
assignment-engine (:4005, Go)

Request flow, file by file

File-by-file request flow

Event-driven microservices · Go · Kafka · Redis

8 — assignment-engine (:4005, Go)

Job: the product's whole thesis. Kafka in, Kafka out; it owns no database.

```
cmd/engine/main.go
├─ candidates.NewFinder(redisURL)    internal/candidates/redis.go
├─ offers.New(client, dedupTTL)       internal/offers/store.go
├─ events.NewProducer(brokers)        internal/events/kafka.go
├─ engine.Engine{...}                 internal/engine/engine.go
├─ go watcher.Run(ctx)                internal/engine/watcher.go
├─ go consume(donation.created)
└─ go consume(donation.timeout)

— Kafka: donation.created —————
▼
internal/events/kafka.go    Consumer fetches the message
▼
cmd/engine/main.go
├─ parse → events.DonationCreated    internal/events/types.go
├─ eventId read from the BODY, not the header
│   (!) a REPLAYED message carries no headers – reading the
│   header there caused a donation to be assigned twice
└─ dedup: SETNX processed:<eventId>    internal/offers/store.go
    └─ already seen → commit and move on
    └─ on failure → RELEASE the marker, or the retry skips it
▼
internal/engine/engine.go    Handle()
├─ no coordinates? → publish donation.unassigned NOT_GEOCODED
│   └─ announced, never silently dropped
▼
offerRound()                — continued below —
```

The scoring round — `internal/engine/engine.go` `offerRound()`

The heart of the system. Everything above exists to get here safely.

```

offerRound()
├─ internal/scoring/urgency.go
│   RadiusLadderKm(category)  HIGH 5→12→25→50 km
│                               MED 5→10→20→35
│                               LOW 5→8→12→20
│   ResponseTimeout(category) 90s / 3min / 5min
├─ for each radius on the ladder:
│   └─ internal/candidates/redis.go Nearby()
│       GEOSEARCH agents:live BYRADIUS <r> km ASC
│       ▶ skips anyone whose agent:<id>:alive has expired
│       ▶ reads caps + load for each hit
│   └─ internal/scoring/score.go RankAgents()
│       └─ HARD FILTERS, before any scoring:
│           ▶ excluded (already declined/ignored)
│           ▶ !Available
│           ▶ !Capabilities.Handles(category)
│       └─ ScoreAgent() for everyone left:
│           DistanceScore 1 - dist/radius      × 0.35
│           CategoryScore compatibility.go     × 0.25
│           LoadScore     1/(1+pending)        × 0.20
│           RatingScore    rating/5             × 0.15
│                               total out of 0.95
│       └─ sort: score ▶ then distance ▶ then agentId
│           (the id tie-break makes it reproducible)
│   └─ nobody eligible? → next radius up the ladder
│   └─ TopN(3)
│   └─ internal/offers/store.go
│       donation:<id>:offer      the ranks + full breakdowns
│       ZADD offers:deadlines <expiresAt> <donationId>
│   └─ internal/events/kafka.go Publish donation.assigned
│       ▶ all three offered IN PARALLEL, one event
└─ ladder exhausted —————
    publish donation.unassigned with a reason:
        NO_AGENTS_FOUND • NO_ELIGIBLE_AGENTS • ALL_AGENTS_EXHAUSTED

```

The HTTP side and the timeout loop

```
— HTTP side (what the agent's UI calls) —————
internal/api/handlers.go
GET  /offers/:id          the offer as the agent sees it
POST /offers/:id/accept   SETNX donation:<id>:claimed
                             ▶ winner 200 · losers 409
POST /offers/:id/reject   adds them to donation:<id>:declined
```

```
— background goroutine —————
internal/engine/watcher.go  Run()
| ZRANGEBYSCORE offers:deadlines 0 <now>
| publish donation.timeout
└ engine.HandleTimeout()
  | Offers.Declined() → the exclusion set
  | (!) without it, a re-score finds the IDENTICAL top 3 and
  | offers it back to the people who just ignored it
└ offerRound() again, at the next radius
```